

Lecture 2
Volatile, Thread-safe Collections, java.util.concurrent

S.l. Dr. Ing. Radu MIRON
radu.miron@aut.utcluj.ro

Java Concurrent Programming History

Thread, Runnable, synchronized, wait()/notify(), join() (Java 1 - 1996)

Executors and Future variables (*java.util.concurrent*) (Java 5 - 2004)

CompleatableFuture (Java 8 - 2014)

Virtual Threads (Java 21 - 2023)

Volatile, Thread-safe collections

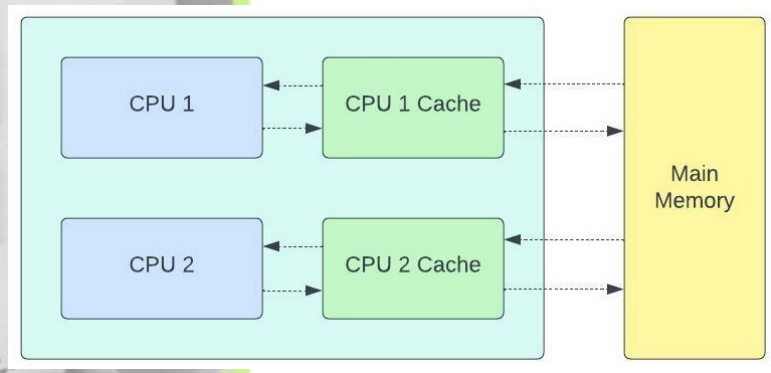


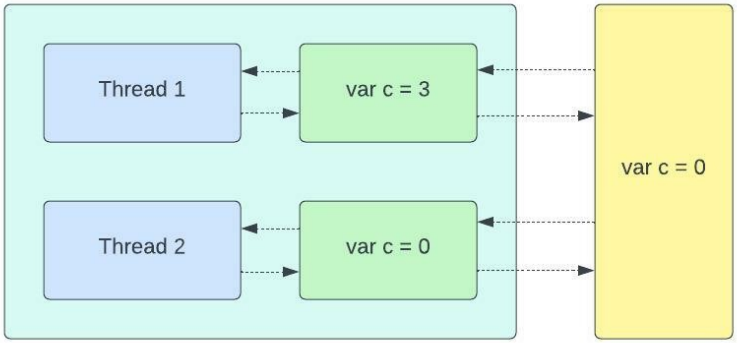
DOCUMENTATION:
https://www.geeksforgeeks.org/volatile-keyword-in-java/
https://www.baeldung.com/java-synchronized-collections

Keyword “**volatile**”

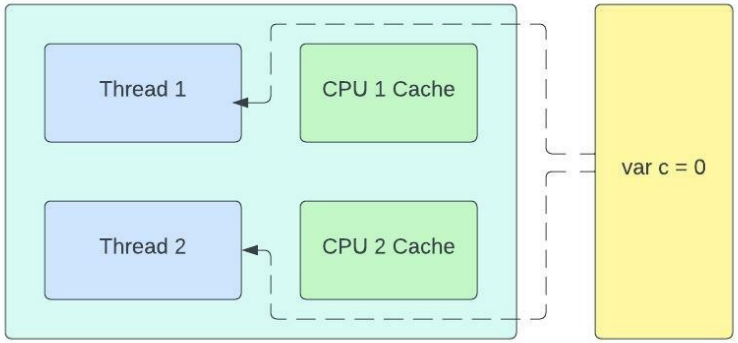
It causes a variable to be stored in the main memory area shared by all processors (cores).

Exemple – the design pattern *Sigleton* (demo).





variabila *non-volatile*



variabila *volatile*



Thread-safe Collections

A code sequence is thread-safe if it works with shared data in a way that can guarantee data safety when it is accessed by multiple threads at the same time.

-> unsafe collections

```
List<Integer> unsafeList = new ArrayList<>();
Queue<Long> unsafeQueue = new LinkedList<>();
Set<String> unsafeSet = new HashSet<>();
Map<Integer, Task> unsafeMap = new HashMap<>();
// about all common collections that you know
```

-> safe synchronized collections

```
List<Integer> threadSafeList =
                        Collections.synchronizedList(new ArrayList<>());
Set<Task> threadSafeSet =
                        Collections.synchronizedSet(new HashSet<>());
Map<Integer, String> theadSafeMap =
                        Collections.synchronizedMap(new HashMap<>());
```



Thread-safe collections

-> thread-safe concurrent collections

```
List<Integer> list = new CopyOnWriteArrayList<>();
Set<String> set = new CopyOnWriteArraySet<>();
Queue<Task> queue = new ConcurrentLinkedQueue<>();
Map<Long, String> map = new ConcurrentHashMap<>();
```

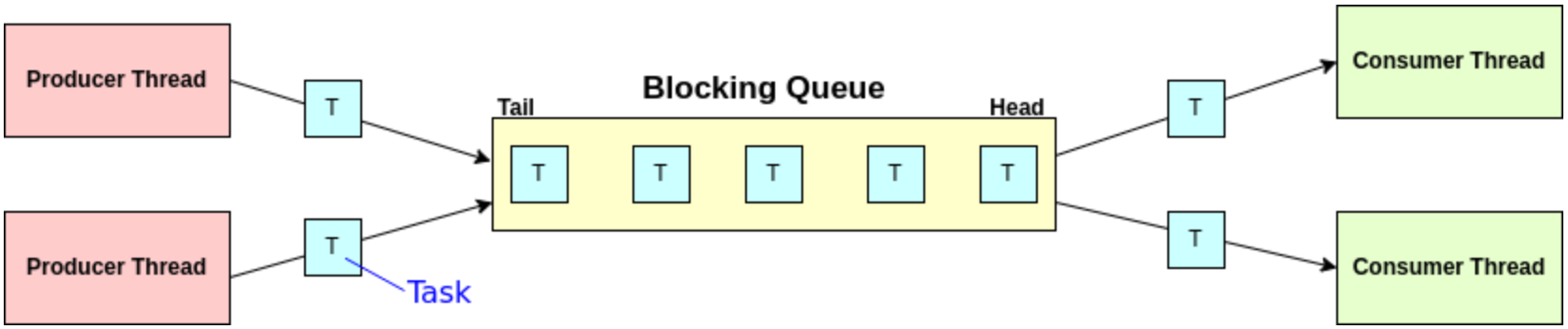
-> Code example:
Producer - consumer with **ArrayBlockingQueue**.

Concurrent collections are the preferred ones since they are faster!

Design Pattern: Producer-Consumer

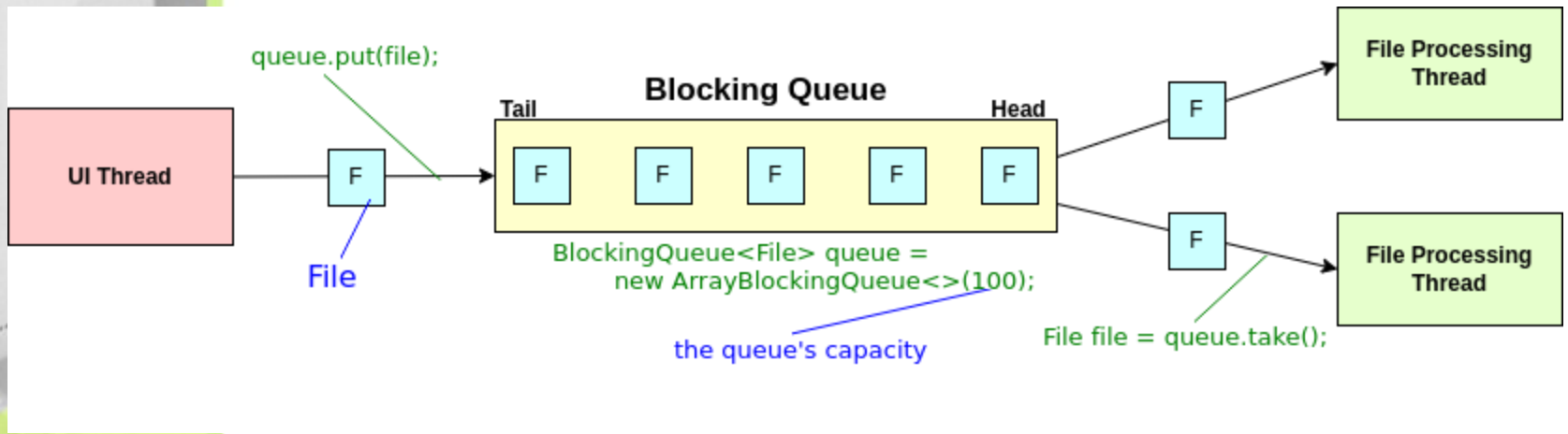
DOCUMENTATION:
https://jenkov.com/tutorials/java-concurrency/producer-consumer.html
https://medium.com/@karthik.jeyapal/system-design-patterns-producer-consumer-pattern-45edcb16d544

The producer-consumer design pattern is a concurrency-specific design model in which one or more producer threads create items, such as data or events, which are then placed in a queue to be consumed by one or more consumer threads. The items represent pending tasks. By separating task creation from task execution, the system gains the flexibility to manage the number of threads simultaneously involved in generating or executing these tasks.



Producer-Consumer Example

Implement a Java application with a graphical user interface that allows users to select image files in order to convert them into text files. The file selection and file processing operations must be decoupled by means of an *ArrayBlockingQueue*.



OCR library: tess4j -> tesseract

```
<dependency>
   <groupId>net.sourceforge.tess4j</groupId>
   <artifactId>tess4j</artifactId>
   <version>5.10.0</version>
</dependency>
```

See lecture repository

The *java.util.concurrent* package

DOCUMENTATION:
https://www.baeldung.com/java-concurrent-locks
https://www.baeldung.com/java-semaphore
https://www.baeldung.com/java-cyclic-barrier
https://www.baeldung.com/java-countdown-latch

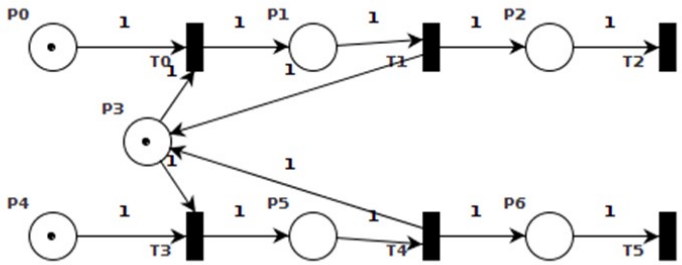
Lock → mutual exclusion

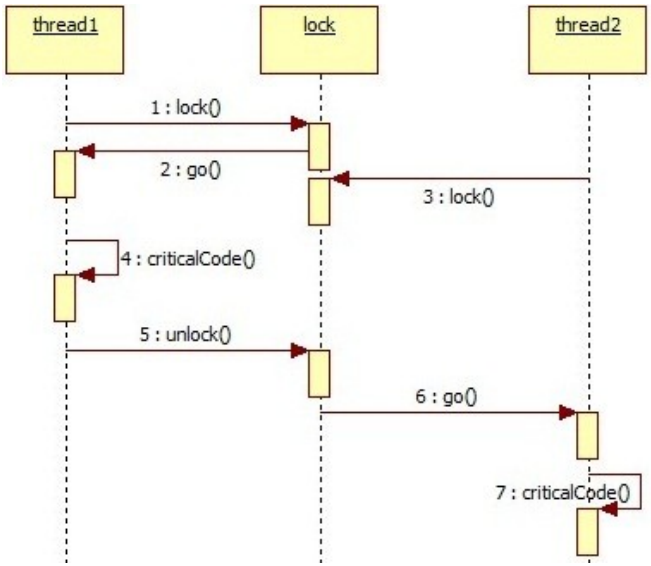
```
Lock l =
   new ReentrantLock();

l.lock();
try{
      // critical region
} finally {
      l.unlock();
}

-----------------------------

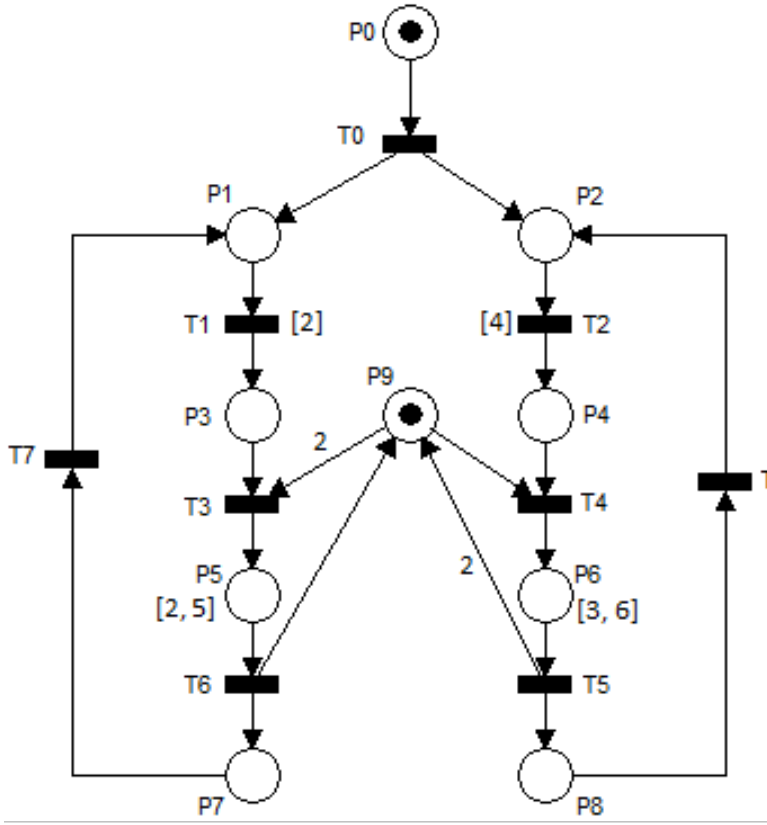
if (l.tryLock()) {
      try {
            // critical region
      } finally {
            l.unlock();
      }
} else {
      // alternative code
}
```



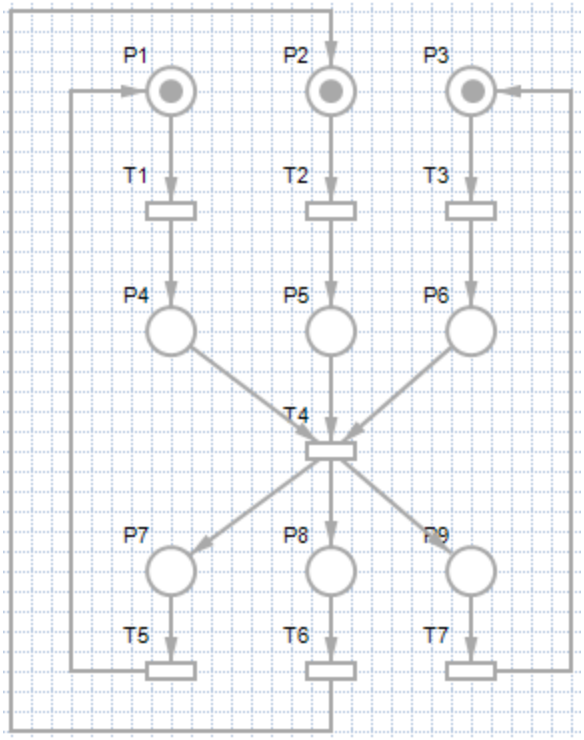


Semaphore → mutex + more

```
Semaphore s = new Semaphore(1);
-----------------------------------------------------------
// Thread 1
try {
   s.acquire(2);
   // critical region
} catch (InterruptedException ignored) {
} finally {
   s.release(1);
}
-----------------------------------------------------------
// Thread 2
try {
   s.acquire();
   // critical region
} catch (InterruptedException ignored) {
} finally {
   s.release(2);
}
```



CyclicBarrier

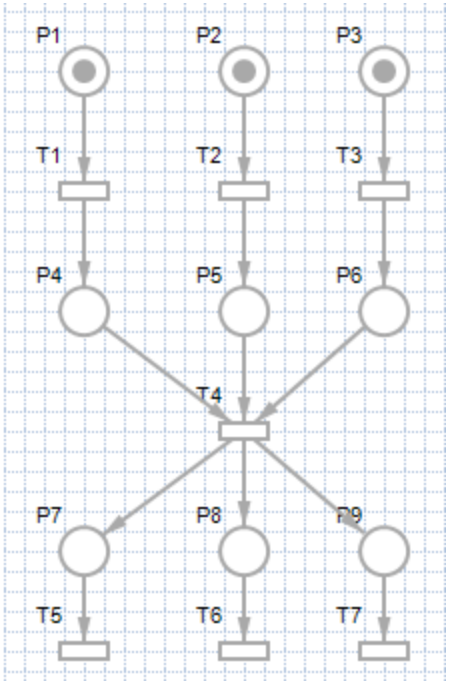



```
//this CyclicBarrier instance should be the same for all 3 threads
CyclicBarrier cyclicBarrier = new CyclicBarrier(3);

//the next code snippet is in all three threads
while (true) { // it works every time
    activityBeforeSynch();
    cyclicBarrier.await();
    activityAfterSynch();
}
```



CountDownLatch




```
//this CountDownLatch instance should be the same for all 3 threads
CountDownLatch countDownLatch = new CountDownLatch(3);

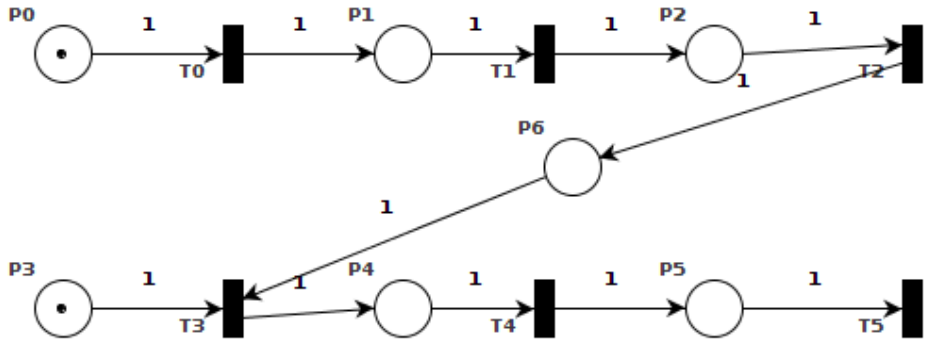
//the next code snippet is in all three threads
activityBeforeSynch();
countDownLatch.countDown();
countDownLatch.await();
activityAfterSynch();
```



CDL vs. CB

- ***CountDownLatch*** **is not reusable.**
- ***CyclicBarrier*** **counts threads, whereas *CountDownLatch* does not necessarily do so; its counter can be decremented independently by any thread, even multiple times.**

wait/notify - Semaphore

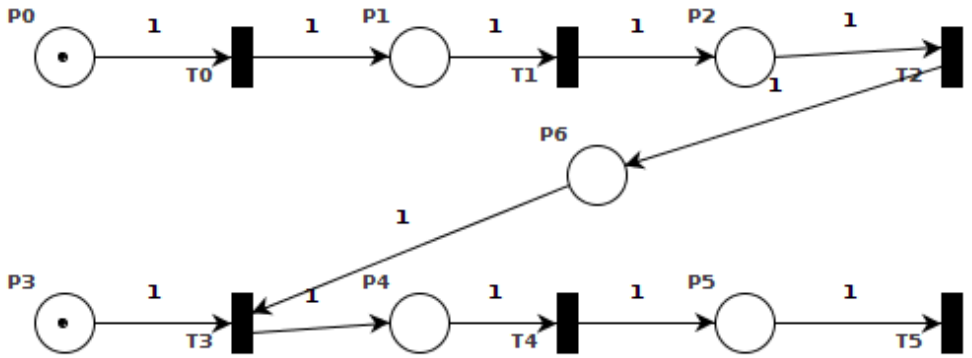



```
//this Semaphore instance should be the same for both threads
Semaphore semaphore = new Semaphore(0);

//------------waiter thread-----------------
semaphore.acquire(); // this is similar to wait
activityAfterNotify();

//------------notifier thread---------------
activityBeforeNotify();
semaphore.release(); // this is similar to notify
```

wait/notify - CountDownLatch




```
//this CountDownLatch instance should be the same for both threads
CountDownLatch countDownLatch = new CountDownLatch(1);

//------------waiter thread-----------------
countDownLatch.await(); // this is similar to wait
activityAfterNotify();

//------------notifier thread---------------
activityBeforeNotify();
countDownLatch.countDown(); // this is similar to notify
```



The end!